

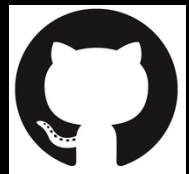
¿QUIEN SOY?

Samuel Isaí Muñoz Pereira

Contacto:

- samuelmp7@gmail.com
- 2828200000312@ingenieria.usac.edu.gt

Soy estudiante de décimo semestre, apasionado por Linux, el hacking y la seguridad informática. Me interesa especialmente el área de Redes, Cloud y DevOps, en mi tiempo libre disfruto experimentar con electrónica. Disfruto mucho la música y la ciencia ficción



SmillerMP



Samuel Muñoz



Samuel Muñoz



SmillerMP





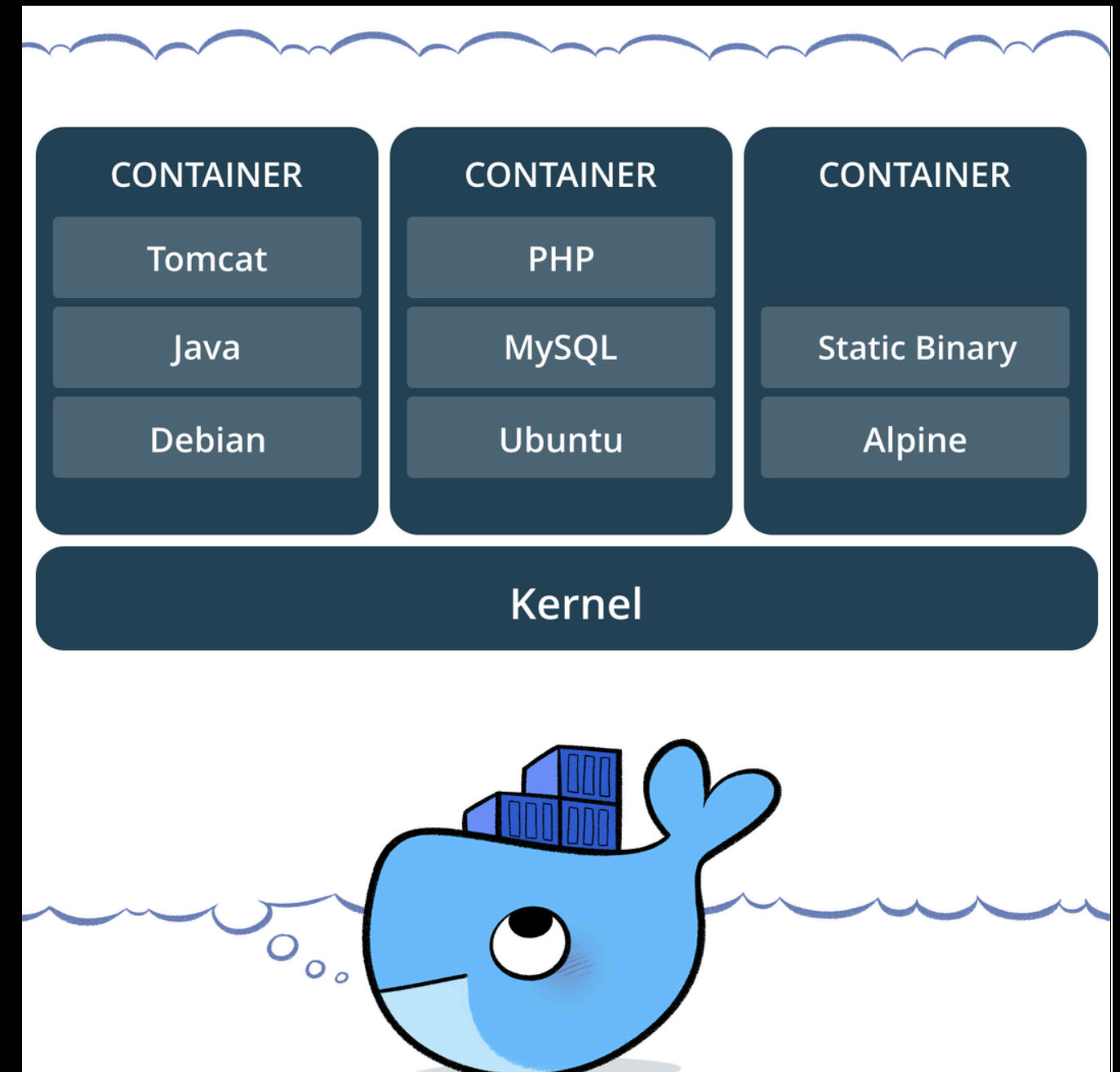
ORACLE, API Y DOCKER

SAMUEL ISAÍ MUÑOZ PEREIRA

¿QUÉ ES DOCKER?

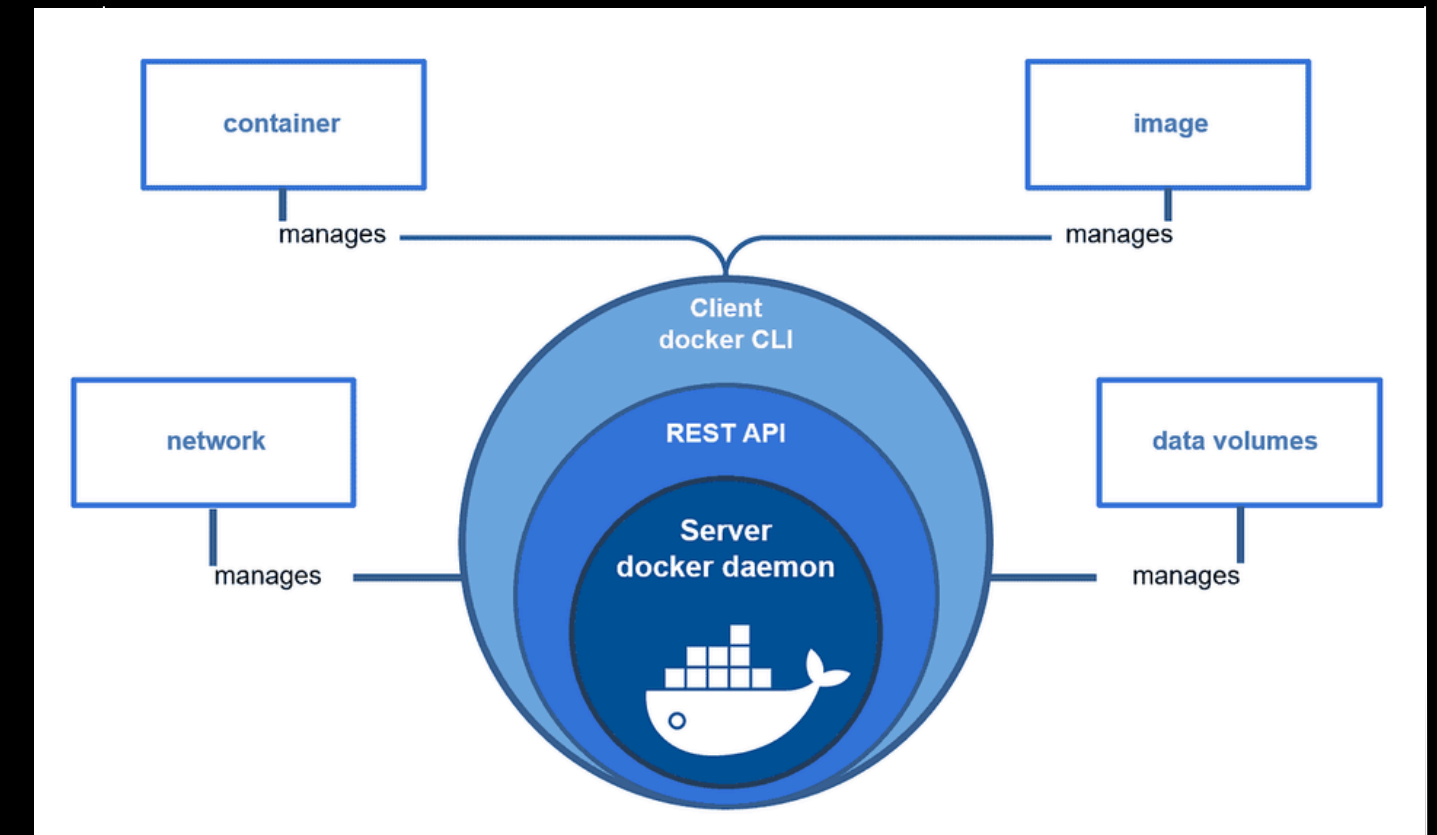
“DOCKER ES COMO UNA CAJA DONDE METES TU APP + TODO SU ENTORNO, Y ESA CAJA FUNCIONA IGUAL EN TU PC, EN EL SERVIDOR O EN LA NUBE.”

- Basado en el kernel de Linux
- Uso de namespaces (aislamiento)
- Uso de cgroups (control de recursos)
- Virtualización a nivel de sistema operativo
- Comparte el kernel del host
- Contenedores en lugar de máquinas virtuales
- Ejecución de procesos aislados



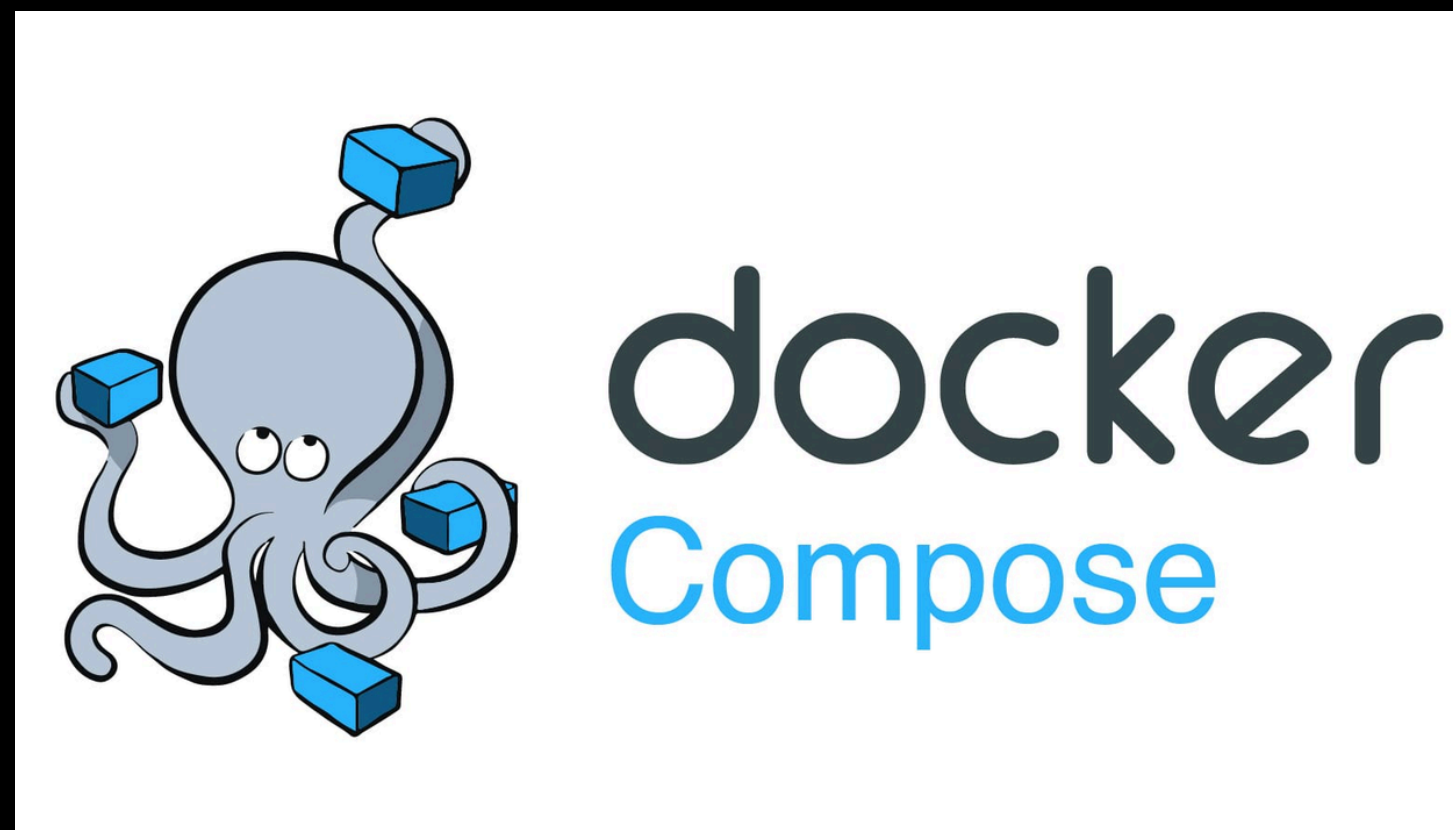
¿POR QUÉ DOCKER?

- Evita el “en mi máquina funciona”
- Entornos consistentes entre desarrollo y producción
- Despliegues más rápidos
- Configuración reproducible
- Aislamiento de aplicaciones
- Facilita el trabajo en equipo
- Menos conflictos de dependencias
- Escalabilidad sencilla
- Integración con CI/CD



DOCKER COMPOSE

- Define múltiples servicios en un solo archivo
- Levanta toda la arquitectura con un comando
- Manejo automático de redes entre servicios
- Configuración centralizada (docker-compose.yml)
- Orquestación básica de contenedores
- Dependencias entre servicios (depends_on)
- Uso de volúmenes para persistencia
- Variables de entorno integradas
- Ideal para entornos de desarrollo



- `docker compose up -d` → levanta en segundo plano
 - `docker compose down` → detiene y elimina contenedores
 - `docker compose build` → construye las imágenes
 - `docker compose logs -f` → logs en tiempo real
 - `docker compose exec <servicio> <comando>` → ejecutar dentro de un contenedor
 - `docker compose restart` → reinicia servicios
 - `docker compose stop` → detiene sin eliminar
 - `docker compose start` → inicia servicios detenidos
-
- Se usa el nombre del servicio como hostname
 - Todos los servicios están en una misma red por defecto
 - No necesitas configurar IPs manualmente
 - `depends_on` controla el orden de inicio
 - Los cambios en el código requieren `rebuild` (`build`) si no usas volúmenes
 - Puedes mapear puertos para acceder desde fuera
 - Los volúmenes permiten persistir datos
 - Puedes usar variables de entorno (`.env`)
 - `docker-compose.yml` define toda la arquitectura



COMANDOS IMPORTANTES

